

1 MDS-WebUtil

1.1 Purpose

The library WebUtil provides helper functions that you can use to perform http and https calls with little code.

1.2 Requirements

SP16

1.3 Code example

```
String result = new CommonHttpClient()  
    withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))  
    communicate("https://api.predic8.de/shop/products/", null);  
System.out.println(result);
```

This example calls the URL <https://api.predic8.de/shop/products/> using GET and outputs the result to the command line.

1.4 API reference



You cannot use classes and functions that are not documented when you use the MDS license.

1.4.1 CommonHttpClient

The class CommonHttpClient is immutable. You can create an instance and then, if necessary, overwrite the default values by means of the withXX() methods.

You call the URL by using one of the communicateXX() methods.

1.4.1.1 CommonHttpClient(): CommonHttpClient

Creates an HTTP client that uses system defaults. This client can be used with HTTP or HTTPS. For HTTPS the defaults allow a connection without a truststore. Optionally you can set a truststore by using one of the withTrustStoreFileXX() methods.

1.4.1.2 withRequestSettings(ICheckedConsumer<HttpURLConnection, IOException> settingSetter): CommonHttpClient

Allows to set parameters on the HttpURLConnection like the HTTP verb (GET, POST, DELETE, etc) or other header values like 'Content-Type'

Input:

settingSetter: callback for setting configuration values on the HttpURLConnection

Return:

new instance with applied settings

Sample:

```
new CommonHttpClient()
    .withRequestSettings(urlCon -> {
        urlCon.setRequestMethod("POST");
        urlCon.setRequestProperty("Content-Type", "application/json");
    });
```

1.4.1.3 withBasicAuthData(String user, String password): CommonHttpClient

Allows to specify basic authentication credentials

Input:

user: the user

password: the password

Return:

new instance with applied settings

Sample:

```
new CommonHttpClient()
    .withBasicAuthData("MyUser", "MyPassword");
```

1.4.1.4 withCookieSupport(CookieManager cookieManager): CommonHttpClient

Adds cookie support to the request. Captures cookies and sets them on following requests

Input:

cookieManager: cookie manager

Return:

new instance with applied settings

Sample:

```
new CommonHttpClient()
    .withCookieSupport(cookieManager);
```

1.4.1.5 withTruststoreFile(String truststoreFilePath, String truststorePassword): CommonHttpClient

This method allows to specify a trust store and a truststore password.

Input:

truststoreFilePath: local absolute file path to truststore

truststorePassword: password of truststore

Return:

new instance with applied settings

1.4.1.6 withTruststoreFileStreamProvider(ICheckedSupplier<InputStream, IOException> truststoreFileInputStreamSupplier, String truststorePassword): CommonHttpClient

Allows to specify a truststore and a truststore password. The ICheckedSupplier allows, for example, the usage of system paths.

Input:

truststoreFileInputStreamSupplier: supplier of an InputStream to the truststore content

truststorePassword: password of truststore

Return:

new instance with applied settings

1.4.1.7 withCheckHostNames(): CommonHttpClient

Allows to enable the check of the host name of a certificate against the called target. Default is OFF.

Return:

new instance with applied settings

1.4.1.8 withSslSocketFactory(SSLSocketFactory sslSocketFactory): CommonHttpClient

Allows to use an externally created SSL factory with settings like truststore.

Input:

sslSocketFactory: externally created SSL socket factory

Return:

new instance with applied settings

1.4.1.9 withOverwrittenSecureSocketProtocol(String secureSocketProtocol): CommonHttpClient

This method allows to overwrite the secure socket protocol. The default used is "TLS".

Input:

secureSocketProtocol: the secure socket protocol to be used

Return:

new instance with applied settings

1.4.1.10 withOverwrittenTruststoreType(String truststoreType): CommonHttpClient

This method allows to overwrite the truststore type. The default is `KeyStore.getDefaultType()`.

Input:

truststoreType: the truststore type to be used

Return:

new instance with applied settings

1.4.1.11 withOverwrittenTruststoreAlgorithm(String truststoreAlgorithm): CommonHttpClient

This method allows to overwrite the truststore algorithm. The default is `TrustManagerFactory.getDefaultAlgorithm()`.

Input:

truststoreAlgorithm: the truststore algorithm to be used

Return:

new instance with applied settings

1.4.1.12 communicate(String urlString, String payload):

CommonHttpClient

Executes a HTTP / HTTPS request.

Input:

urlString: URL to communicate with; can be an HTTP or HTTPS URL

payload: payload or NULL: In case of GET there is no payload allowed

Return:

result from URL

Throws:

IOException: if any IO error occurred

HttpException: if the server responds with HTTP code >= 400

Sample:

```
new CommonHttpClient()  
    .withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))  
    .communicate("https://api.predic8.de/shop/products/", null);
```

1.4.1.13 T communicate(String urlString, String payload, ICheckedFunction<HttpURLConnection, T, IOException> resultTransformer)

Executes a HTTP / HTTPS request.

Input:

urlString: URL to communicate with; can be an HTTP or HTTPS URL

payload: payload or NULL: In case of GET there is no payload allowed

resultTransformer: callback to extract and transform result from HttpURLConnection. This callback can also access and extract result headers.

Return:

result created by resultTransformer

Throws:

IOException: if any IO error occurred

1.4.1.14 void communicateAsStream(String urlString, CheckedConsumer<OutputStream, IOException> requestProcessor, CheckedConsumer<HttpURLConnection, IOException> responseProcessor)

Executes a HTTP / HTTPS request and allows streaming of data.

Input:

urlString: URL to communicate with; can be an HTTP or HTTPS URL

requestProcessor: optional callback to provide payload in streaming manner, can be NULL

responseProcessor: mandatory callback to provide result processing in streaming manner

Throws:

IOException: if any IO error occurred

1.4.2 HttpException

This exception transports the response code from a http request and the error message.

1.4.2.1 getResponseCode(): int

Gets the http response code

Return:

http response code

1.4.2.2 getMessage(): String

Returns the http error result

Return:

http error result